BDL 2025 Smart Contract Coursework Report

Exam Number: B299013

October 23, 2025

1 Detailed Contract Design Decisions

This section outlines the key design choices implemented in the Vickrey Auction smart contract (`B299013.sol`).

1.1 How are bids kept private during the bidding phase?

Bid privacy is achieved using a **commit-reveal scheme**, a standard technique to mitigate front-running. The auction proceeds in two distinct phases governed by timestamps:

1. **Commit Phase (until `biddingEndTime`):** Bidders do not submit their actual bid amount. Instead, they submit a cryptographic commitment (a hash) of their bid amount combined with a secret nonce (a random, securely generated value). This is calculated off-chain as `keccak256(abi.encodePacked(_bidAmount, _nonce))`. They also send their maximum possible bid value (or higher) as a deposit (`msg.value`) during the `commitBid()` function call. The hash hides the true bid amount from other bidders and observers during this phase.

2. **Reveal Phase (between `biddingEndTime` and `revealEndTime`):** Bidders call the `revealBid()` function, submitting their original bid amount and nonce. The contract recalculates the hash using these revealed values (`keccak256(abi.encodePacked(_bidAmount, _nonce))`) and verifies it against the commitment submitted earlier. Only if the hashes match is the bid considered valid.

This ensures that bid values remain confidential until the bidding phase is closed, preventing participants from adjusting their bids based on others' revealed values during the bidding window.

1.2 How is the payment sent to the seller?

The contract employs the **pull-over-push payment pattern** (Lecture 04, Slide 12-14) for enhanced security and robustness. Instead of the contract automatically "pushing" funds to the seller upon finalization, the process is:

1. **Finalisation (`finalisedAuction()`):** When the auction concludes, the contract calculates the winning price (the second-highest valid bid, or the reserve price if only one bid or the second bid is below reserve). This calculated amount is then credited to the seller's balance within a dedicated mapping: `mapping(address => uint256) public pendingWithdrawals`. The actual ETH is not transferred at this stage.

2. **Withdrawal (`withdrawFunds()`):** The seller (or any other user with a pending balance, like refunded bidders) must explicitly call the `withdrawFunds()` function. This function checks their balance in `pendingWithdrawals`, sets it to zero (following the Checks-Effects-Interactions pattern to prevent re-entrancy), and then transfers the ETH to their address (`msg.sender`).

This pattern avoids potential issues associated with push payments, such as hitting gas limits if the recipient is a contract with a complex fallback function, or unexpected failures if the recipient rejects the payment, which could otherwise block the finalization process (DoS/Griefing, Lecture 04, Slide 11). It shifts the responsibility (and gas cost) of the final transfer to the recipient.

1.3 How is it guaranteed that a seller / bidder cannot cheat?

Several mechanisms prevent cheating:

- **Shill Bidding Prevention:** The seller is explicitly prohibited from bidding on their own auction via the check `require(msg.sender != auction.seller)` within the `commitBid()` function. While a seller could technically use a separate address, this prevents direct bidding from the registered seller address.
- **Commit-Reveal Scheme:** As described above, this prevents bidders from seeing others' bids during the bidding phase and adjusting their own bid accordingly. It also prevents bidders from changing their bid amount after committing; they can only reveal the amount corresponding to the hash they initially submitted.
- **Single Bid Commitment:** The check `require(bids[_auctionId][msg.sender].commitment == bytes32(0), "Bid has already been committed.")` in `commitBid()` ensures that each address can only submit one commitment per auction, preventing a bidder from submitting multiple hidden bids and revealing only the most advantageous one.
- **NFT and Fund Escrow:** The contract acts as a neutral escrow. The NFT is transferred to the contract using `safeTransferFrom` when the auction starts (`startAuction()`), ensuring the seller cannot withdraw it during the auction. Bidder deposits (`msg.value`) are held by the contract until the auction is finalised. The `finalisedAuction()` function then handles the logic for transferring the NFT to the winner and making funds available for withdrawal (to the seller and refunded bidders) according to the auction rules. This prevents either party from absconding with both the NFT and the funds.
- **Deposit Requirement:** Bidders must deposit an amount greater than or equal to their intended bid (`require(bid.deposit >= _bidAmount)` in `revealBid()`). This ensures they have the funds to back their bid if they win.
- **Re-entrancy Protection:** Withdrawal functions (`withdrawFunds()`, `claimLosingBid()`) implement the Checks-Effects-Interactions pattern by setting the user's balance (`pendingWithdrawals` or `bid.deposit`) to zero before executing the external ETH transfer (`.call{value: ...}` or adding to `pendingWithdrawals`). This prevents re-entrancy attacks where a malicious contract could repeatedly call back into the withdraw function before the balance is updated (Lecture 04, Slide 21-27).

1.4 How are ties settled?

Ties occur if multiple bidders reveal the exact same highest bid amount. The contract settles ties deterministically based on the **commit block number**. The logic within `revealBid()` checks:

```
else if (_bidAmount == auction.highestBid) {
    if (bid.commitBlock < auction.highestBidCommitBlock) {
        // Current bid wins the tie break
        auction.secondHighestBid = auction.highestBid;
        auction.highestBidder = payable(msg.sender);
        auction.highestBidCommitBlock = bid.commitBlock;
```

```
    } else {
        // Old bid committed earlier or in the same block,
        // current bid is second highest
        auction.secondHighestBid = _bidAmount;
    }
}
```

This means if two bids have the same value, the bid that was included in an earlier block (`bid.commitBlock < auction.highestBidCommitBlock`) wins. `block.number` is chosen over `block.timestamp` because timestamps can be manipulated to some extent by miners, whereas block numbers are strictly sequential and harder to manipulate for tie-breaking purposes (Lecture 04, Slide 75-76).

1.5 Are bidders allowed to bid multiple times on the same item or not, and how is this handled?

No, bidders are **not allowed to bid multiple times** in the same auction using the same address. This is enforced by the `commitBid()` function with the check: `require(bids[_auctionId][msg.sender] == bytes32(0), "Bid has already been committed.")`. This line verifies that the `commitment` field for the calling address (`msg.sender`) in the specific auction (`_auctionId`) is still its default zero value (`bytes32(0)`). If a commitment already exists (meaning the bidder has already called `commitBid()` for this auction), the requirement fails, and the transaction reverts, preventing a second commit.

1.6 What data type/structures do you use and why?

The contract utilizes several key data structures:

- **`struct Auction` and `struct Bid`:** Structs are used to group related data logically. `Auction` bundles all information pertaining to a single auction (seller, NFT details, deadlines, status, winner info), while `Bid` groups data related to a specific bidder's participation in an auction (commitment hash, deposit, commit block, reveal status). This improves code readability and organization compared to managing individual variables for each property.
- **`mapping(uint256 => Auction) public auctions`:** A mapping links a unique auction ID (`uint256 auctionCounter`) to its corresponding `Auction` struct. Mappings provide efficient O(1) lookup based on the key (the auction ID).
- **`mapping(uint256 => mapping(address => Bid)) public bids`:** A nested mapping is used to store bid information. The outer key is the auction ID, and the inner key is the bidder's address. This structure efficiently retrieves a specific bidder's `Bid` struct for a given auction (e.g., `bids[auctionId][bidderAddress]`).
- **`mapping(address => uint256) public pendingWithdrawals`:** This mapping stores the amount of ETH owed to each address (sellers or refunded bidders) as part of the pull payment pattern. It allows for efficient tracking and retrieval of withdrawable balances.
- **`IERC721 public immutable nftContract`:** An immutable state variable stores the address of the NFT contract. Using `immutable` saves gas compared to a regular state variable because the value is set in the constructor and baked directly into the contract bytecode, rather than being stored in a storage slot. The `IERC721` interface type allows interaction with the standard NFT functions like `ownerOf` and `safeTransferFrom`.

- **Basic Types:** Standard types like `address payable` (for ETH transfers), `uint256` (for amounts, IDs, timestamps, block numbers), `bytes32` (for hashes/commitments), and `bool` (for flags like `active`, `finalised`, `revealed`) are used for their respective purposes. Solidity `^0.8.0` is used, which includes built-in checks for integer overflow/underflow (Lecture 04, Slide 97).

1.7 Other Design Decisions

- **Events:** The contract emits events (`AuctionStarted`, `BidCommitted`, `BidRevealed`, `AuctionFinalised`, `FundsWithdrawn`) to log significant actions. This facilitates off-chain monitoring and indexing of contract activity.
- **IERC721Receiver Implementation:** The contract inherits from `IERC721Receiver` and implements the `onERC721Received` function. This is necessary to allow the contract to securely receive the NFT via `safeTransferFrom` when an auction is started, acting as the escrow agent.
- **Reserve Price:** The `reservePrice` allows the seller to set a minimum acceptable price. Bids below this value are ignored during winner determination in `revealBid()`. If no valid bids meet or exceed the reserve price, the NFT is returned to the seller during finalization.
- **Timestamp-based Deadlines:** `block.timestamp` is used to define `biddingEndTime` and `revealEndTime`. While timestamps have some miner manipulability, they are generally considered acceptable for defining time periods in Solidity, unlike using them for critical randomness or tie-breaking logic.

2 Gas Evaluation

(Provide a detailed gas evaluation. You can use a table here.)

Function	**Gas Cost (Approx.)**
Contract Deployment	[Gas cost]
`startAuction()`	[Gas cost]
`commitBid()`	[Gas cost]
`revealBid()` (losing bid)	[Gas cost]
`revealBid()` (new highest bid)	[Gas cost]
`finalisedAuction()`	[Gas cost]
`claimLosingBid()`	[Gas cost]
`withdrawFunds()`	[Gas cost]

Table 1: Gas costs of primary contract interactions.

2.1 Gas Fairness

(Discuss if some roles pay more gas. For example, `finalisedAuction` does a lot of work. Also, the first person to reveal a new highest bid pays slightly more gas than subsequent bidders because they write to more storage slots: `highestBid`, `highestBidder`, `secondHighestBid`, `highestBidCommitBlock`.)

2.2 Efficiency Techniques

(Mention any techniques you used to save gas, e.g., using `immutable` for `nftContract`, choosing appropriate data types, Solidity version 0.8+ optimizations.)

3 Potential Vulnerabilities and Hazards

3.1 Re-entrancy

(Explain how your `withdrawFunds` and `claimLosingBid` functions prevent re-entrancy by following the Checks-Effects-Interactions pattern, specifically by setting `pendingWithdrawals[msg.sender] = 0` or `bid.deposit = 0` before the external ETH transfer (`.call{value: ...}` or adding to `pendingWithdrawals`). Reference Lecture 04, Slide 27-28.)

3.2 Shill Bidding

(Explain the `require(msg.sender != auction.seller)` check. Acknowledge the limitation that a seller could still use a different wallet address.)

3.3 Timestamp/Block Variable Manipulation

(Explain why `block.number` (`commitBlock`) was used for the tie-breaker instead of `block.timestamp`, as timestamps can be manipulated by miners (Lecture 04, Slide 75-76, 88). Acknowledge that `block.timestamp` is used for deadlines, which is a common practice but carries a minor risk if precise timing is critical.)

3.4 Integer Overflow/Underflow

(Mention that the contract uses `pragma solidity ^0.8.0;`, and Solidity versions 0.8.0 and above have built-in checks for arithmetic overflow and underflow, mitigating this risk without needing external libraries like SafeMath (Lecture 04, Slide 97).)

4 Design Tradeoffs and Choices

(This is a good place to discuss your tie-breaking rule. You chose "earlier commit block wins." Tradeoffs: Fairly deterministic, prevents reveal-phase gas wars. Downside: If two valuable bids are in the same block, the tie is arbitrary based on internal transaction ordering within the block, though still deterministic. Contrast with "first to reveal wins" - creates incentives for gas wars during reveal. Contrast with "random" tie-breaking - difficult/insecure to implement on-chain.)

(Also, discuss the "pull-over-push" pattern: Tradeoff: Enhanced security (prevents re-entrancy in caller, avoids external call failures blocking execution) and potentially better gas fairness vs. poorer user experience (requires users to send one extra transaction (`withdrawFunds` or `claimLosingBid`) to retrieve their ETH).)

5 Analysis of Fellow Student's Contract

(Analyze your peer's contract here. Include their contract address.)

5.1 Security Analysis

(Did they prevent re-entrancy? Shill bidding? Did they use `block.timestamp` insecurely for randomness or tie-breaking? Did they handle integer overflow/underflow if using ^{0}.8.0?$Didtheycorrectlyimplem$
$reveal?Didtheyusepull - over - push?$)

5.2 Efficiency and Fairness

(Is their gas usage reasonable compared to yours? Are there obvious gas optimizations missed? Is their tie-breaking rule fair? Who pays the gas for key operations like finalization or refunds?)

5.3 Discovered Vulnerabilities

(Detail any specific vulnerabilities found (e.g., re-entrancy possibility, insecure randomness, logic errors) and explain step-by-step how you would exploit them.)

6 On-chain Interaction Details

6.1 My Deployed Contract Address (B299013)

Address: [YOUR_CONTRACT_ADDRESS_HERE]

6.2 Peer's Deployed Contract Address

Address: [PEER_CONTRACT_ADDRESS_HERE]

6.3 My Transaction Hashes

(List your key transactions, e.g., starting an auction, bidding on your peer's auction.)

- **Start Auction (My Contract):** [TX_HASH_HERE]
- **Commit Bid (Peer Contract):** [TX_HASH_HERE]
- **Reveal Bid (Peer Contract):** [TX_HASH_HERE]
- **Finalise (My Contract):** [TX_HASH_HERE]
- **Withdraw Funds/Claim Refund (My or Peer Contract):** [TX_HASH_HERE]